DSA Pattern Identification

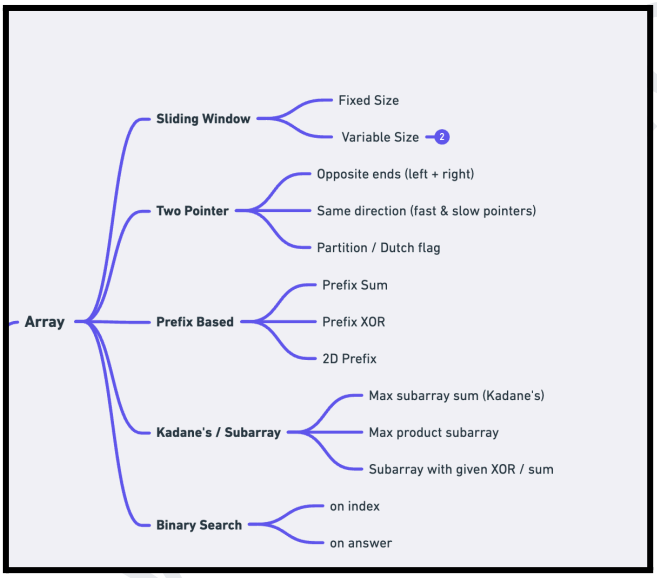
DSA Patterns: [DSA Patterns X CodeWithNishchal]

Identifying Data Structure in Question is an art! That you will learn in this sheet completely.
NOTE: First Learn DS in depth then learn the identification.

This guide will teach you 2 Types of DS Identification:

- By looking at the questions
- By looking at the brute force solution

Array



1. Two Pointer Identification

- **MAJOR TRICK**

Sorted Array + pair/triplet + sum/target = Two Pointer (← →)

1. Sorted Array + pair/triplet + sum/target = **Two Pointer (← →)**

2. Sorted Array + reverse / palindrome / compare both ends = **Two Pointer (← →)**

3. Sorted Array + duplicate removal / value shift/ in-place modify = **2 Pointer (→ →)**

4. Two sorted arrays + merge / intersect / common elements → **Two Pointer (→ →)**

CodeWithNishchal

2. Sliding Window Identification

- **Core rule:**

Array/ String + Subarray /Substring (contiguous) + (longest / shortest / count / max / / min / avg) = Sliding Window

Fixed window

"size = k" → Fixed Sliding Window

Variable window

Condition given (≤ k, ≥ k, unique, distinct, etc.)
→ Variable Sliding Window

Identifications :

1. Array + subarray + maximum sum (no condition) + positive & negative + no window constraint = **Kadane's**
2. Array + subarray + exact sum = k → **Prefix Sum + HashMap** (optional note: works even better when negatives exist)

ARRAY PATTERNS DECISION TREE

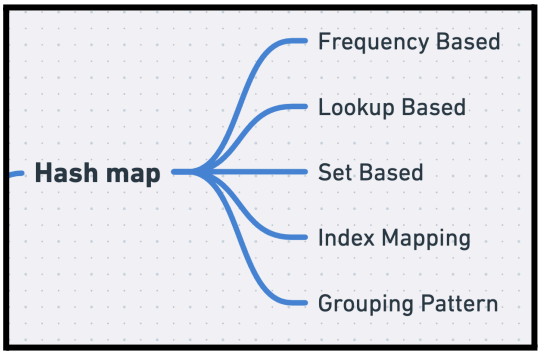
```
START
│
├── Is it SUBARRAY / SUBSTRING (contiguous)?
│   │
│   ├── YES →
│   │   │
│   │   ├── Only maximum subarray sum?
│   │   │   → Kadane's Algorithm
│   │   │
│   │   ├── Fixed size (k)?
│   │   │   → Sliding Window (Fixed)
│   │   │
│   │   ├── Condition given (≤k, ≥k, distinct, unique...)?
│   │   │   → Sliding Window (Variable)
│   │   │
│   │   └── Count / exact sum = k?
│   │       → Prefix Sum + HashMap
│   │
│   └── NO ↓
│
```

CodeWithNishchal

```
├── Is it PAIR / TRIPLET + SUM / TARGET?
│   │
│   ├── YES →
│   │   │
│   │   ├── Sorted (or can sort)?
│   │   │     → Two Pointer (← →)
│   │   │
│   │   └── Not sorted?
│   │         → HashMap (Two Sum style)
│   │
│   └── NO ↓
│
├── Is it BOTH-END COMPARISON?
│   │   (palindrome / reverse / greedy shrink)
│   │
│   ├── YES → Two Pointer (← →)
│   │
│   └── NO ↓
│
├── Is it IN-PLACE MODIFY?
│   │   (remove / shift / overwrite elements)
│   │
│   ├── YES → Two Pointer (→ →)
│   │         works on any sorted array
│   │
│   └── NO ↓
│
└── Are there TWO ARRAYS?
    │   (merge / intersect / common elements)
    │
    ├── YES → Two Pointer (→ →)
    │         (arrays usually sorted or can be sorted)
    │
    └── NO → Try other patterns (Stack / DP / Binary Search)
```

CodeWithNishchal

3. Hashmap Identification



HashMap = memory for fast decisions

We already know that : **sorted + pair + Target sum → Two Pointer**
But in the case of :
unsorted + pair + sum → HashMap

CORE HASHMAP IDENTIFICATION

1. Count / frequency / occurrences → HashMap

2. Need fast lookup (O(1)) — "does X exist?"→ HashMap

3. Duplicate / unique / distinct → HashMap

4. Replace nested loop ($O(n^2) \rightarrow O(n)$) → HashMap

5. Subarray + sum = k → Prefix Sum + HashMap

6. Grouping problems → HashMap

7. unsorted + pair / complement — "have I seen target - X before?" → HashMap

CodeWithNishchal

4. Searching Identification

CORE RULE:

If search space is sorted OR can be reduced → Binary Search

SEARCHING IDENTIFICATION

1. Sorted array + Target element+ find/search/index = **Binary Search**

2. Answer in numeric range + isValid() check = **Binary Search on Answer**

3. Array+ unsorted + structured(peak / mountain/ rotated sorted) = **Binary Search**

4. Not sorted + no special structure = **Linear Search**

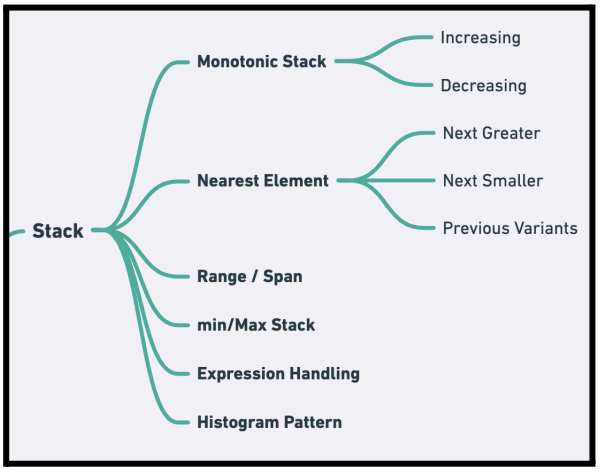
5. No random access (Linked List) = Linear Search

SEARCHING DECISION TREE

```
├── No random access (linked list, stream)?
│  → Linear Search
│
└── Random access allowed (array/matrix)?
│
├── UNSORTED?
│  ├── No special structure → Linear Search
│  ├── Structured (peak / mountain / rotated sorted) → Binary Search
│
└── SORTED?
├── Find element / index / target → Binary Search
├── First / last occurrence → Binary Search
├── Sorted matrix (properly ordered) → Binary Search
├── Answer in range + isValid() → Binary Search on Answer
├── "minimum/maximum satisfying condition" → Binary Search on Answer
└── First TRUE / Last FALSE (monotonic condition) → Binary Search
```

CodeWithNishchal

5. Stack Identification



Core Rule :

1. **If order matters + you need nearest/previous/next → Stack**
2. Whenever you need to remember the LAST SEEN element to make a decision about the CURRENT element = **Stack**

KEYWORD TRIGGERS:

1.Next / Previous / Nearest + Greater / Smaller → Monotonic Stack [IMP]
2. Balanced parentheses / pair matching → Stack
3. Expression evaluation (infix/prefix/postfix) → Stack
4. Largest rectangle / area under histogram → Monotonic Stack
5. Undo / back / reverse order operations → Stack
6. Span / range problems (Stock Span, daily temperatures) → Monotonic Stack

Identification with BRUTE FORCE:

If a brute-force solution uses **two nested loops**, where the **inner loop depends on the outer loop index (i)** in specific ways, it is a strong indicator for stack optimization.

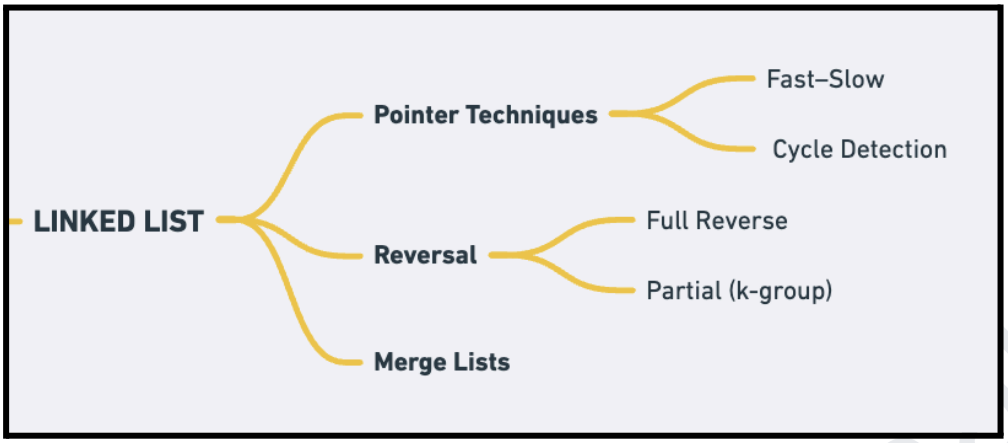
Here i comes from the outer loop

- Common patterns:
  - **Inner loop runs from 0 to i** → Next Smaller Element (Left)
  - **Inner loop runs from i to 0** → Next Greater Element (Left)
  - **Inner loop runs from i to n** → Next Greater Element (Right)
  - **Inner loop runs from n to i** → Next Smaller Element (Right)

So if any of the four patterns are there in your brute force solution => Definitely, this problem can be solved by the Stack Data Structure.

CodeWithNishchal

6. Linked List Identification



Most of the time, the question itself will tell you that it's a linked list question.

Core Identification:

1. Reverse (full / k-group, in-place) → Linked List

2. Cycle detection / loop → Linked List (Fast-Slow Pointer)

3. Find middle / nth from end → Linked List (Fast-Slow Pointer)

3. Navigation back/forward / Undo/Redo → Linked List (Doubly)

4. Merge sorted Seq / streams/List → Linked List

5. Design custom data structure (LRU, browser history) → Linked List + HashMap

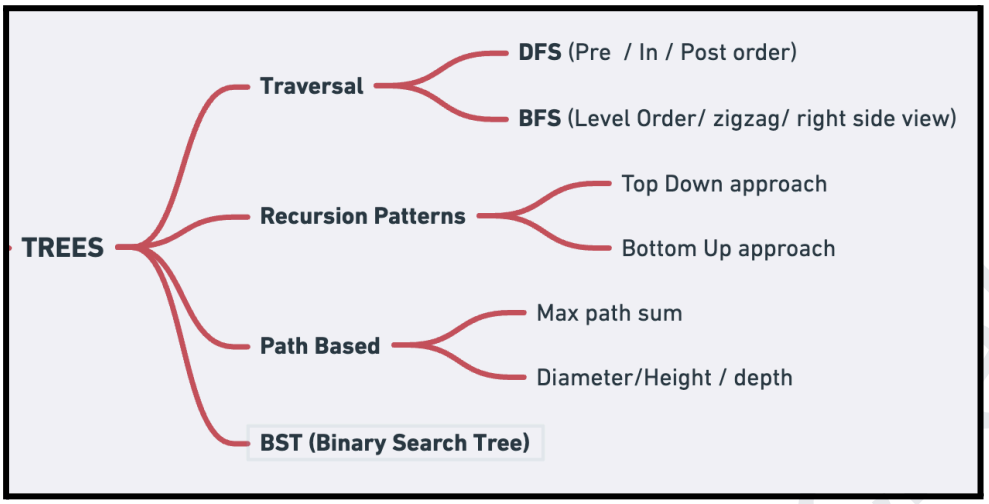
6. Array causes shifting (insert/delete) → Linked List

BRUTE FORCE SIGNAL:

Array + frequent insert/delete/shift → Linked List

CodeWithNishchal

7. Tree Identification



CORE RULE:

1. Path / depth / subtree / recursion → DFS
2. Level / distance / nearest / shortest → BFS

CORE Identification:

1. Rooted Tree + depth/path/height/leaf + max/min/count **= DFS**
2. Tree + level / k distance / nearest / zigzag / leftmost **= BFS**
3. LCA / ancestor → DFS (Target-based upward movement) **= DFS + parent tracking**

DFS TRIGGERS:

1. Height / depth / diameter of tree → DFS
2. Root to leaf path / path sum → DFS
3. Subtree match / subtree sum → DFS
4. LCA / ancestor tracking → DFS + parent tracking
5. Inorder / preorder / postorder → DFS

BFS TRIGGERS:

1. Level order traversal → BFS
2. Zigzag / left side / right side view → BFS
3. K distance from node / nearest node → BFS
4. Minimum depth → BFS
5. Connect next right pointers → BFS

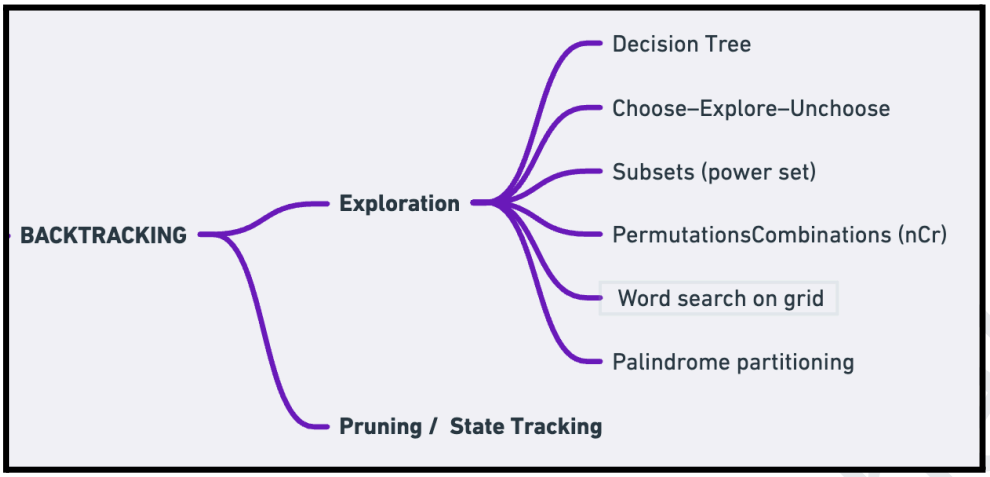
Shortest path in tree (unweighted) → BFS

BRUTE FORCE SIGNAL:

1. Need to explore all paths / subtrees → DFS
2. Need level-by-level processing → BFS

CodeWithNishant

8. Backtracking Identification



CORE RULE:

Try all possibilities + undo choices → Backtracking

MAIN IDENTIFICATION:

Multiple choices at each step + need ALL answers → Backtracking

CORE TRIGGERS:

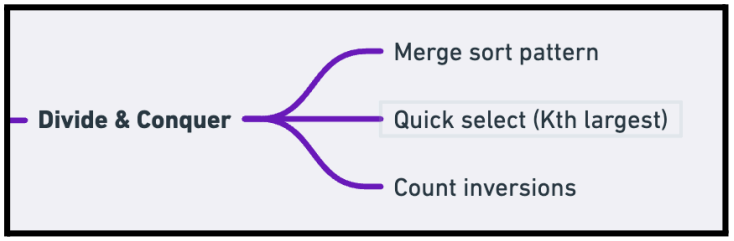
1. Generate all combinations / subsets → Backtracking
2. Generate all permutations → Backtracking
3. “All possible ways” / “print all” → Backtracking
4. Decision at each step (pick / not pick) + **need ALL answers** → Backtracking
5. Constraint + validation (N-Queens, Sudoku, Word Search) → Backtracking
6. Decision tree where **every path = one valid answer** → Backtracking

ONE High Level Rule:

Trigger	Pattern
All subsets / permutations	Backtracking
Count / max / min (optimization only)	DP/greedy
Does path exist (single path, no need all paths)	DFS / BFS
All paths	Backtracking

CodeWithNishchal

9. Divide & Conquer Identification



CORE:

Divide → Solve → Combine → Divide & Conquer

Remember:

1. Divide & Conquer = recursion on independent parts
2. DP → overlapping subproblems (reuse needed)

CORE IDENTIFICATIONS:

1. Split into left/right halves → **Divide & Conquer**

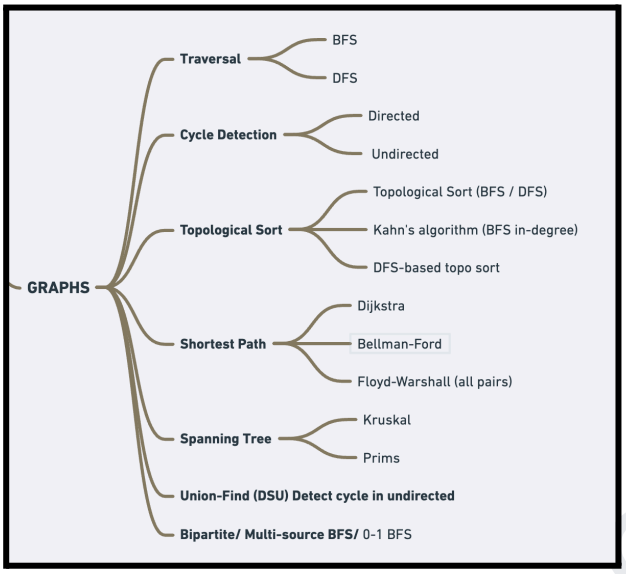
2. Solve subproblems independently/ Combine results → **Divide & Conquer**

3. Recursion on independent parts (no shared work) → **Divide & Conquer**

4. Sorting / searching / inversion type problems → Divide & Conquer

CodeWithNishchal

10. Graph Identification



CORE:

- Grid/Graph + connected components + count/reachable → BFS/DFS
- Grid/Graph + shortest path + unweighted → BFS
- Order dependencies + cycle detection → Topological Sort

BFS triggers:

- Grid/Graph + shortest path + **weighted edges** → Dijkstra (not plain BFS)
- Grid/Graph + **minimum steps** / **minimum moves** → BFS
- Shortest path + weighted + non-negative edges → Dijkstra

DFS triggers:

- Grid/Graph + **all paths**→ DFS
- Grid/Graph + **exists a path** → DFS
- Cycle detection (undirected) → DFS / Union Find
- Cycle detection (directed) → DFS / Topological Sort
- Grid/Graph + **island counting** / **flood fill** → DFS

Topological Sort triggers:

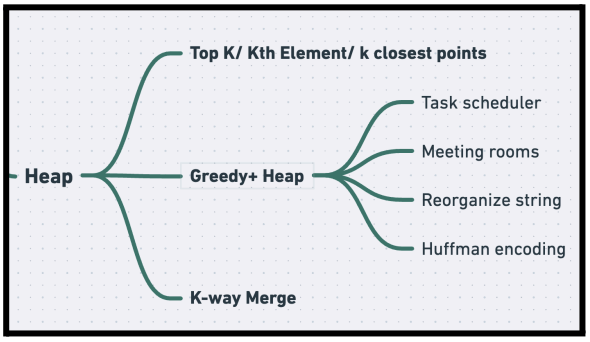
- **Prerequisites** / **task scheduling** / **build order** → Topological Sort
- DAG + **ordering** → Topological Sort

Union Find triggers:

- **Dynamically connecting** components → Union Find
- **Are X and Y connected?** → Union Find

CodeWithNishchal

11. Heap Identification



1. Heap is not for sorting everything. It's for getting "top k / best" efficiently
2. **Heap = smart selection (not full sorting)**

HIGH LEVEL RULE

If sorting is too slow and you only need k elements → **Heap**

1. Smallest needed → Min Heap
2. Largest needed → Max Heap
3. Priority Queue = Heap (same thing, different name)

ULTRA SHORT VERSION

1. Top K → Heap
2. Best element repeatedly → Heap
3. Priority → Heap

HEAP IDENTIFICATION

1. Top K / Kth element → Heap
2. Best element repeatedly (min/max) → Heap
3. Priority / scheduling → Heap
4. Stream + maintain top elements → Heap
5. Merge k sorted lists/arrays → Heap
6. Median of stream / running median → Two Heaps (Min + Max)

Do NOT use Heap when:

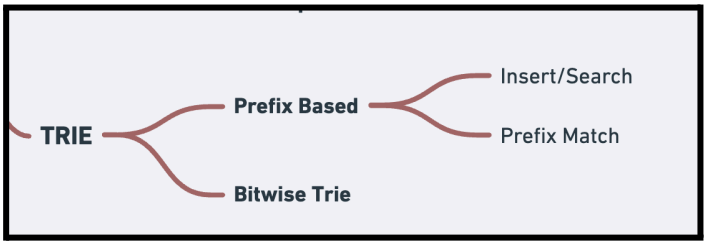
- Need full sorted array → use sorting
- Need exact lookup → HashMap
- Need range/window → Sliding Window

BRUTE FORCE SIGNAL:

If brute force sorts the array repeatedly to find min/max → Heap If brute force uses nested loop to find Kth element → Heap

CodeWithNishchal

12. Trie Identification



CORE RULE:

Whenever you need **prefix-based search / matching** on strings **= Trie**

HIGH LEVEL RULE:

- Need to search **by prefix** → Trie
- Need to match **multiple strings** together → Trie
- Many strings + repeated prefix queries → Trie

TRIE IDENTIFICATION:

1. Prefix based queries / search → Trie
2. Autocomplete / suggestions → Trie
3. Pattern matching with wildcard → Trie
4. Dictionary based lookup / replacement → Trie
5. Search multiple words on same grid → Trie
6. "Starts with" / "count words with prefix" → Trie
7. Longest common prefix → Trie (optional, sorting also works)

DO NOT use Trie when:

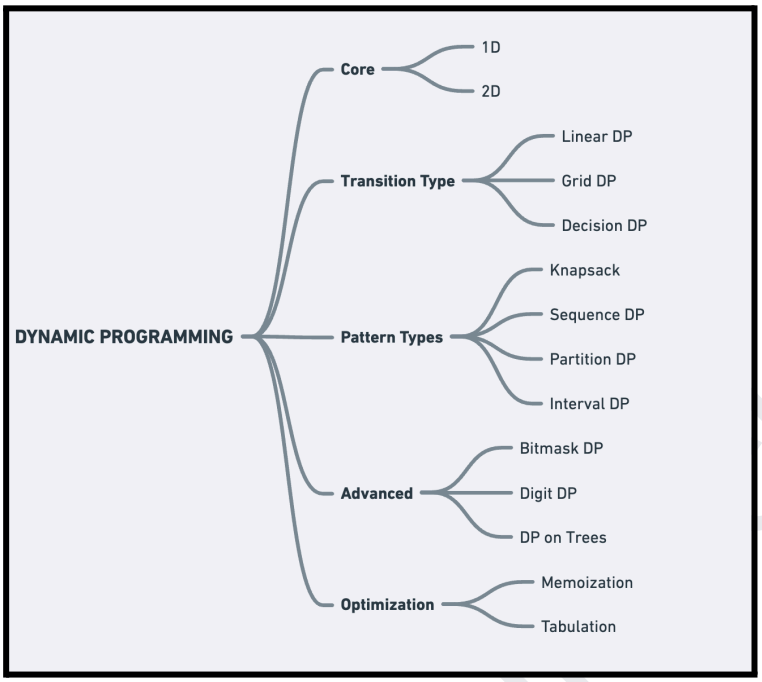
- Need exact lookup only → HashMap
- Single string search → KMP / sliding window
- No prefix relationship → HashMap is enough

CONFUSION ZONE:

Trigger	Pattern
Prefix search / autocomplete	Trie
Exact word lookup	HashMap
Substring search	Sliding Window / KMP
Multiple word search on grid	Trie + DFS
Single word search on grid	DFS/Backtracking

CodeWithNishchal

13. Dynamic Programming (DP) Identification



CORE RULE :

- Overlapping subproblems + optimal substructure → **DP**
- If same subproblem is solved multiple times → **DP over D&C**

HIGH LEVEL RULE:

- "How many ways" / "is it possible" / "minimum cost" → DP
- Need ALL answers → Backtracking (not DP)

DP IDENTIFICATION

STEP 1 — Can a problem be defined by a STATE?

- NO → Not DP
- YES → go to Step 2

STEP 2 — Does the same STATE repeat?

- NO → Divide & Conquer
- YES → DP ✅

CodeWithNirmal

STEP 3 — What defines the STATE?

State	DP Type
Single index i	Linear DP
(index, capacity/constraint)	Knapsack DP
Range (left, right)	Interval DP
Coordinates (row, col)	Grid DP
Tree node	Tree DP
Bitmask of choices	Bitmask DP
Two sequences (i, j)	String DP (LCS/Edit Distance)

KEYWORD TRIGGERS by DP type:

Linear DP:

1. Longest increasing subsequence → Linear DP
2. Max sum subarray (with conditions) → Linear DP
3. Climbing stairs / fibonacci style → Linear DP
4. House robber style → Linear DP

Knapsack DP:

1. Pick/not pick + capacity/budget constraint → Knapsack
2. "Minimum cost to reach target" → Knapsack
3. "Can we make sum = k" → Knapsack
4. Coin change / unbounded knapsack → Knapsack

Interval DP:

1. Matrix chain multiplication → Interval DP
2. Burst balloons → Interval DP
3. Palindrome partitioning → Interval DP
4. "Minimum cost to merge" → Interval DP

CodeWithNishchal

Grid DP:

1. Unique paths / minimum path sum → Grid DP
2. Dungeon game / cherry pickup → Grid DP
3. Obstacles in grid + reach destination → Grid DP

String DP:

1. LCS / Edit Distance → String DP
2. Edit distance → String DP
3. Wildcard / regex matching → String DP
4. Longest palindromic subsequence → String DP

Bitmask DP:

1. Visit all nodes / cities exactly once → Bitmask DP
2. TSP / assignment problems → Bitmask DP
3. Small n (≤20) + subsets of elements → Bitmask DP

DO NOT use DP when:

- Need all answers → Backtracking
- Subproblems independent → D&C
- Simple greedy works → Greedy

CONFUSION ZONE:

Trigger	Pattern
Count / max / min / possible	DP
All answers / print all	Backtracking
Independent subproblems	D&C
Overlapping subproblems	DP
Pick/not pick + ALL answers	Backtracking
Pick/not pick + count/max/min	DP (Knapsack)

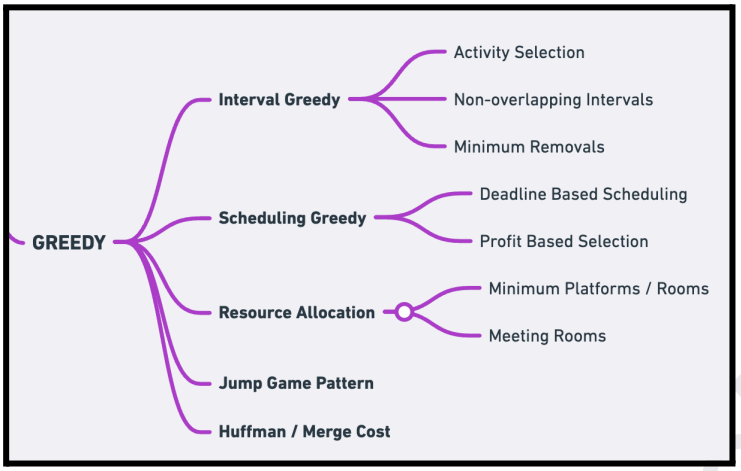
CodeWithNishchal

High level Tree for DP identification

```
Can a problem be defined in terms of a STATE?
|
+--> NO → Not DP
|
+--> YES
      |
      |-- Same STATE solved multiple times?
            |
            +--> NO → Recursion / Divide & Conquer
            |
            +--> YES → ✓ DYNAMIC PROGRAMMING
                      |
                      |-- State defined by:
                      |
                      |-- Single index (i)?
                      |      → Linear DP
                      |
                      |-- (index, capacity / constraint)?
                      |      → Knapsack DP
                      |
                      |-- Range (left, right)?
                      |      → Interval / Partition DP
                      |
                      |-- Coordinates (row, col)?
                      |      → Grid DP
                      |
                      |-- Tree node?
                      |      → Tree DP
                      |
                      |-- Decisions (pick / not pick)?
                             → Knapsack / Subsequence DP
```

CodeWithNishchal

14. Greedy Algorithm Identification



CORE RULE:

Make the **locally best choice** at each step → trust it leads to global best → Greedy No backtracking, no revisiting decisions

---

HIGH LEVEL RULE:

- DP tries ALL choices → picks best
- Greedy picks ONE best choice → moves on
- If greedy choice never needs to be undone → Greedy over DP

GREEDY IDENTIFICATION:

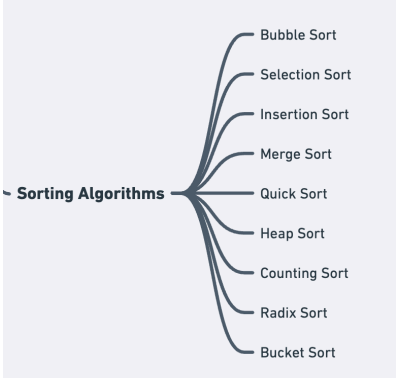
1. "Minimum number of coins / jumps / intervals" → "Minimum coins / jumps" → Greedy (only if standard greedy works, else DP)
2. Activity selection / interval scheduling → Greedy
3. "Maximum events / tasks you can attend" → Greedy
4. Huffman coding / minimum cost tree → Greedy
5. Always pick largest / smallest first → Greedy
6. Job scheduling with deadlines → Greedy
7. "Minimum platforms / rooms needed" → Greedy

BRUTE FORCE SIGNAL:

Sort + pick best option at each step → Greedy

CodeWithNishchal

15. Sorting Identification



WHEN TO USE WHICH:

Algorithm	Use When
Bubble Sort	Don't use it!!
Selection Sort	Don't use it!!
Insertion Sort	Array is almost sorted / very small (n ≤ 20)
Merge Sort	Need stable sort + count inversions + linked list sorting
Quick Sort	General purpose + in-place + average case matters
Heap Sort	Need in-place + worst case O(n log n) guaranteed
Counting Sort	Small integer range (0 to k, k is small)
Radix Sort	Large integers / fixed digit numbers
Bucket Sort	Uniformly distributed floating point numbers

KEYWORD TRIGGERS:

1. "Sort linked list" → Merge Sort (no random access)
2. "Count inversions" → Merge Sort
3. "Almost sorted array" → Insertion Sort
4. "Sort with limited memory" → Heap Sort
5. "Numbers in range 0 to k" → Counting Sort
6. "General sorting, no constraint" → Quick Sort
7. "Need stable sort" → Merge Sort
8. "Integers with fixed digits" → Radix Sort

CodeWithNishchal

CONFUSION ZONE:

Situation	Pick
Linked List sorting	Merge Sort
Almost sorted	Insertion Sort
Small range integers	Counting Sort
Need stable + general	Merge Sort
In-place + general	Quick Sort
Worst case guaranteed	Merge Sort / Heap Sort
Floating point uniform data	Bucket Sort

ONE LINE MEMORY AID:

- **Small / almost sorted** → Insertion Sort
- **General purpose** → Quick Sort
- **Stable / linked list / inversions** → Merge Sort
- **Integer range known** → Counting / Radix Sort
- **Worst case matters** → Merge Sort

NOTE:

1. **Please do not memorize all of this.** First, master the data structures, then use these identification techniques.
2. **These patterns/tricks work in around 95% of cases.** There will always be a few exceptions, but such problems are rare in interviews.
3. **Please do not copy or distribute this content** over social media without my permission. It is created for your learning only.

Show Some Love By Following CodeWithNishchal on social media:

- Instagram: https://www.instagram.com/codewithnishchal/
- LinkedIn: https://www.linkedin.com/in/nishchal-muradia/
- YouTube: https://www.youtube.com/@CodeWithNishchal
- Watch My Latest Time Complexity Series: Time Complexity
- Join My Telegram group: CodeWithNishchal X Telegram

CodeWithNishchal